

KOMPLETNA DOKUMENTACJA PROJEKTOWA

Pełna nazwa przedsięwzięcia: System organizacji szkoły pływania

Nazwa kodowa (kryptonim): SwimSync

Skład zespołu: * Wojciech Osowski (Lider Zespołu)

- Emilia Wojtowicz
- Mateusz Strózek

1. WIZJA I ZAKRES SYSTEMU

Planowane jest stworzenie systemu wspierającego organizację zajęć na basenie. System ma ułatwiać zapisy, planowanie grafiku oraz zarządzanie zajęciami z instruktorami i torami basenowymi.

Kluczowe cele projektu:

- Umożliwienie kursantom samodzielnych zapisów online.
- Optymalizacja obłożenia torów basenowych (zapobieganie konfliktom).
- Dynamiczne zarządzanie dostępnością instruktorów.
- Szybkie informowanie o zmianach (np. odwołanie zajęć).

Zakres realizowanych usług:

- Moduł rejestracji i kont użytkowników (klient, instruktor, admin).
- Interaktywny grafik zajęć z podziałem na tory i poziomy zaawansowania.
- System powiadomień e-mail o zmianach w harmonogramie.
- Panel administratora do zarządzania zasobami (basen, kadra).

Ograniczenia (czego nie realizujemy):

- Systemu płatności online (w tej wersji płatności odbywają się na miejscu).
- Integracji z systemami bramkowymi na basenie.

2. ARCHITEKTURA LOGICZNA

System będzie działał w środowisku webowym, opierając się na następujących technologiach:

- **Frontend:** Aplikacja React.js – responsywny interfejs dla użytkowników końcowych i administracji.
- **Backend:** Python – obsługa walidacji, logiki przydziału torów i harmonogramowania.

- **Baza Danych:** MySQL (w fazie prototypowania silnika zastąpiona elastycznym, plikowym systemem JSON) – przechowywanie danych o użytkownikach, zapisach, instruktorach i dostępności torów.
- **Komunikacja:** Protokół HTTP/HTTPS dla API oraz integracja z zewnętrznym serwerem SMTP dla powiadomień.

3. GŁÓWNE FUNKCJE (EPIKI I HISTORYJKI)

[EPIKA 1] Zarządzanie kursantami i zapisami

- Jako kursant, chcę zapisać się na zajęcia online, aby zarezerwować miejsce w wybranej grupie.
- Jako kursant, chcę widzieć moje nadchodzące lekcje, aby nie zapomnieć o zajęciach.
- Jako kursant, chcę móc z odpowiednim wyprzedzeniem anulować swoją obecność na zajęciach, aby zwolnić miejsce na torze dla innych.
- Jako kursant, chcę przeglądać swoją historię odbytych zajęć, aby śledzić swoje postępy w uczestnictwie.
- Jako instruktor, chcę po zakończonych zajęciach odznaczyć w grafiku, kto był obecny, aby administrator miał poprawną dokumentację frekwencji.
- Jako instruktor, chcę mieć dostęp do listy imiennej uczestników moich zajęć, aby móc sprawdzić ich obecność.
- Jako kursant, chcę zapisać się na cały cykl zajęć (np. semestr/miesiąc), aby mieć zagwarantowane miejsce bez konieczności odklikiwania rezerwacji co tydzień.

[EPIKA 2] Zarządzanie kontami i profilami użytkowników

- Jako kursant, chcę założyć i edytować swój profil (w tym wiek i poziom zaawansowania), aby mieć dostęp do grup dopasowanych do moich umiejętności.
- Jako administrator, chcę mieć możliwość przeglądania i edytowania kont użytkowników, aby móc zresetować hasło.
- Jako instruktor, chcę mieć możliwość dodania krótkiego opisu do swojego profilu, aby kursanci wiedzieli o moich kwalifikacjach.

[EPIKA 3] Komunikacja i powiadomienia

- Jako administrator, chcę wysłać masową wiadomość do grupy, aby poinformować o nagłej awarii basenu itp.
- Jako kursant, chcę otrzymywać przypomnienia o zbliżających się zajęciach, aby zminimalizować ryzyko nieobecności.
- Jako kursant, chcę otrzymywać powiadomienia o zmianach w planie, które mnie dotyczą.
- Jako instruktor, chcę otrzymać automatyczne powiadomienie, gdy administrator dokona zmian w moim grafiku (np. zmieni godzinę lub tor), abym nie pomylił miejsca zajęć.

[EPIKA 4] Zarządzanie zasobami i grafikiem

- Jako instruktor, chcę widzieć swój pełny harmonogram na dany tydzień lub miesiąc, aby odpowiednio zaplanować swój czas pracy i dojazdy na basen.
- Jako instruktor, chcę oznaczyć swoją niedostępność, aby system nie planował mi zajęć w tym czasie.
- Jako administrator, chcę tworzyć nowe grupy pływackie (określając ich maksymalną wielkość, wiek i poziom), aby móc zaplanować nowy semestr nauki.
- Jako administrator, chcę mieć wizualny podgląd zajętości wszystkich torów basenowych w danej godzinie.
- Jako administrator, chcę zdefiniować maksymalną pojemność każdego toru basenowego, aby system zablokował możliwość dołączania nowych osób po przekroczeniu bezpiecznego limitu.
- Jako administrator, chcę, aby system technicznie uniemożliwił mi tworzenie konfliktów (przypisanie dwóch różnych grup do tego samego toru w tej samej godzinie, przypisanie instruktora do zajęć w czasie, gdy prowadzi on już inną grupę).

[EPIKA 5] Raportowanie i monitorowanie

- Jako administrator, chcę generować zbiorcze zestawienia frekwencji dla konkretnych grup, aby weryfikować, które zajęcia cieszą się największą popularnością.
- Jako administrator, chcę wygenerować listę przepracowanych godzin dla każdego z instruktorów, aby łatwo obliczyć ich comiesięczne wynagrodzenie.

4. KRYTERIA AKCEPTACJI I SCENARIUSZE TESTOWE

Historyjka 1: Zapisy na zajęcia online Cel: Zarezerwowanie miejsca w wybranej grupie przez kursanta. **Warunki wstępne:** Kursant posiada konto, jest zalogowany, a w systemie istnieją dostępne grupy zajęciowe. **Proces:** System wyświetla listę dostępnych zajęć. Kursant sprawdza termin, godzinę i poziom grupy, po czym wybiera grupę i zatwierdza zapis. System weryfikuje w tle liczbę wolnych miejsc (z uwzględnieniem limitów grupy i toru). **Warunki końcowe:** Kursant zostaje zapisany do grupy, miejsce zostaje zarezerwowane, a kursant otrzymuje potwierdzenie zapisu. **Wymagania niefunkcjonalne:** Operacja zapisu trwa maksymalnie 2 sekundy. System działa poprawnie na urządzeniach mobilnych.

Historyjka 2: Masowa wiadomość do grupy Cel: Poinformowanie o nagłej awarii basenu przez administratora. **Warunki wstępne:** Administrator jest zalogowany. Istnieje grupa docelowa odbiorców. **Proces:** Administrator wybiera grupę, wpisuje treść wiadomości i wysyła komunikat. **Warunki końcowe:** Wszyscy członkowie grupy otrzymują wiadomość, a system zapisuje historię wysyłki w bazie. **Wymagania niefunkcjonalne:** Wiadomość do 500 użytkowników jest wysyłana w czasie poniżej 1 minuty.

Historyjka 3: Przypomnienia o zbliżających się zajęciach Cel: Zminimalizowanie ryzyka nieobecności kursanta. **Warunki wstępne:** Kursant jest aktywnie zapisany na zajęcia odbywające się następnego dnia. **Proces:** System automatycznie weryfikuje harmonogram i wysyła przypomnienie przed terminem zajęć. **Warunki końcowe:** Kursant otrzymuje wiadomość e-mail z

przypomnieniem. **Wymagania niefunkcjonalne:** Powiadomienie jest wysyłane punktualnie o ustalonej porze przy użyciu zadań cyklicznych (CRON).

5. PROJEKTOWANIE INTERFEJSU I PRZEPŁYWU (UI/UX)

Prace projektowe nad interfejsem użytkownika zostały podzielone na dwie iteracje:

Faza Wstępna (Sprint 2):

- Opracowano projekt flow użytkownika w postaci diagramu aktywności, modelującego ścieżki nawigacyjne po aplikacji.
- Przygotowano wizualizację flow w postaci interaktywnego prototypu interfejsu w wersji Low-Fidelity (makiety szkieletowe).

Badania i Finalizacja (Sprint 3):

- Przeprowadzono testy użyteczności prototypu w gronie potencjalnych użytkowników końcowych (kursantów i instruktorów).
- Sporządzono raport z badania interfejsu. Na podstawie zebranego feedbacku wprowadzono niezbędne poprawki w nawigacji oraz układzie formularzy.
- Sfinalizowano ostateczny prototyp UI w wersji High-Fidelity, stanowiący podstawę wizualną dla wdrażanego systemu.

6. ARCHITEKTURA SYSTEMU I PROJEKT TECHNICZNY (UML)

Podczas prac sformalizowano strukturę techniczną aplikacji za pomocą kompleksowej dokumentacji architektonicznej, niezbędnej do implementacji backendu.

Diagramy Sekwencji: Zaprojektowano 3 potężne przepływy sekwencyjne dla kluczowych modułów systemu:

- Zarządzanie Użytkownikami, Infrastrukturą i Grupami (obejmujące skomplikowane procesy edycji profili oraz deklaracji niedostępności).
- Obsługa Zapisów i Rezerwacji z aktywną detekcją konfliktów (overbooking) oraz mechanizmem Soft Delete.
- Realizacja Zajęć i Powiadomienia (zbiorcze wprowadzanie obecności, system raportowania, zadania CRON).

Diagramy Klas: W oparciu o przepływy sekwencyjne opracowano 15 szczegółowych diagramów klas. Wprowadzono architektoniczny podział na warstwy zgodnie ze wzorcem MVC:

- **Warstwa <<UI>> / Boundaries:** Punkty styku użytkownika z aplikacją (np. PortalZapisow, SystemCron).
- **Warstwa <<Control>>:** Mózg operacyjny weryfikujący reguły biznesowe (np. HarmonogramController).

- **Warstwa <<Entity>>:** Struktury danych i podstawowa logika dziedziczna (np. GrupaPływacka, Rezerwacja).
- **Warstwa <<DB>>:** Moduł persystencji izolujący bazę danych od logiki aplikacji.

7. IMPLEMENTACJA BACKENDOWA (KOD I TESTY)

Zgodnie z planem prac końcowych (Backlog Sprintu 3), w pełni zaimplementowano projekt w języku Python.

7.1. Logika Biznesowa (API Reference)

- **Encje i Serializacja:** Wdrożono klasy dziedziczne. Każdy obiekt posiada metody `to_dict()` oraz `from_dict()`, co pozwala na natywną transformację obiektów do formatu JSON bez użycia zewnętrznych bibliotek ORM.
- **Plikowa Baza Danych:** Zaimplementowano klasę `BazaDanych` obsługującą operacje I/O w automatycznie tworzonym katalogu `data/`. Wdrożono operacje odczytu konfliktów czasowych (parsowanie standardu ISO) oraz masowy zapis obiektów (`bulk_insert`).
- **Kontrolery i Reguły Biznesowe:** Zaimplementowano kluczowe mechanizmy ujęte w historyjkach użytkownika, m.in.: weryfikację wolnych miejsc, blokowanie możliwości stworzenia konfliktu w harmonogramie przez administratora, czy odrzucanie próby bezpłatnego anulowania lekcji na krócej niż 24 godziny przed jej startem.

7.2. Testy Jednostkowe i Integracyjne (Pytest)

System został zabezpieczony pakietem zautomatyzowanych testów w środowisku `pytest`.

- **Bezpieczeństwo danych:** Wykorzystano mechanizm wstrzykiwania zależności oraz wbudowaną armaturę `tmp_path`, co gwarantuje, że każdy test tworzy własną, izolowaną bazę danych w katalogu tymczasowym, chroniąc pliki produkcyjne przed modyfikacją.
- **Pokrycie przypadków użycia:** Przetestowano zachowania optymistyczne (Happy Path) oraz brzegowe (Sad Path). Wykonano asercje weryfikujące m.in. poprawne rzucanie błędów `ValueError` przy próbach zapisu do przepełnionej grupy oraz poprawne generowanie komunikatów ostrzegawczych przechwytywanych na standardowym wyjściu strumieniowym systemu (`capsys`).